PYTHON MACHINE LEARNING · ADVANCED MACHINE LEARNING · FINAL ASSESSMENT

Capstone: Your Pinecrest ML Project

The single end-of-kit assessment (Lesson 13.2). Each learner takes one of five Pinecrest problems from a vague request to a model they can defend: a notebook that reruns, a saved model, one to three figures and a model card. Score it with the lesson's eight-row rubric below. This page also holds the instructor answer notes for all five problems, with reference numbers from real runs on the course data: it is for instructors only.

Covers **Modules 7–13** · Chosen **Session 11**, studios **Sessions 12–13** · Submit within **1 week** · Successful: **3+** on every row · Instructor only

THE BRIEF (AS THE INSTRUCTOR SEES IT)

From Lesson 13.2, the request from Pinecrest's operations and marketing teams: "Several teams have asked for 'some machine learning' this year. Please take one, build the best model you honestly can, and tell us whether we should use it, how, and what could go wrong. We will read the model card, not the code, but our data team will rerun your notebook before anything goes live."

PROBLEM	DECISION	DATA, SPLIT, METRIC	TEMPLATE BASELINE
A · Demand forecasting	How much stock of each product to send to each store	`weekly_demand.csv`; train 2024, test 2025; MAE in units	per-product mean 13.419
B · Fraud review queue	Which online payments a two-person team checks (about 1% a day)	`transactions.csv`; train January to May, test June; average precision, precision and recall at capacity	no model 0.008
C · Review triage	Which reviews a product manager reads first	`reviews.csv` without 3-star; stratified 75/25; accuracy and negative recall	always positive 0.745
D · Recommendations	The ten products on each customer's home page	`interactions.csv` + `products_ml.csv`; leave-last-out; hit rate at 10	most popular unowned 0.209

PROBLEM	DECISION	DATA, SPLIT, METRIC	TEMPLATE BASELINE
E · Customer segments	Which message each group of customers receives	`customers_churn.csv`, behavior columns only; no target; silhouette, stability, profiles	loyalty tiers 0.079

The template's baselines, as the lesson prints them:

```
demand    per-product mean       13.419
fraud     no model               0.008
reviews   always positive        0.745
recs      most popular unowned   0.209
segments  existing loyalty tiers 0.079
```

- **Own problems** are allowed if they pass the lesson's three checks (a decision someone could make; data and labels in `data/` without invented labels; not one of the course's existing projects: churn and daily store sales are taken). Approve in writing before Session 12.
- **Deliverables:** `capstone.ipynb` (reruns from a fresh kernel and produces every number and figure in the card), `model_card.md` (one to two pages, from the lesson's skeleton), one to three figures with finding-first titles, `model.skops` and `requirements.txt`, `results.json`; optional `serve.py` with a test. Plus a README (with the fictional-data line and the help used) and the self-scored rubric.
- **Assessed:** the submitted folder against the eight rubric rows. The Session 13 showcase is practice and is not scored; note one strength per learner from it for the written feedback.
- **There is no answer key for learners.** The notes below are for coaching and grading only. Every problem has at least one honest negative result; a careful "do not deploy" or "cannot be told apart" scores exactly as well as a dramatic success (the lesson says so in a callout).

WHERE THE 480 MINUTES GO

MILESTONE (LESSON ESTIMATE)	MINUTES	WHERE IT HAPPENS IN THIS KIT
1 · Choose and plan	30	Session 11 (choice + paragraph, 10) and home; checked in the Session 12 plan clinic
2 · Load, check and split	40	Session 12 studio
3 · Baseline and evaluation harness	30	Session 12 studio
4 · A first pipeline	45	Session 12 studio (started), home (finished)
5 · Improve	90	Home, between Sessions 12 and 13

MILESTONE (LESSON ESTIMATE)	MINUTES	WHERE IT HAPPENS IN THIS KIT
6 · Test once, then analyze the errors	45	Session 13 studio
7 · Explain and review	40	Session 13 studio
8 · Figures and model card	45	Session 13 (the figure and the first two card sections, 20), home (25)
9 · Reproduce, save and review	30	Home, within a week of Session 13
Optional · Serve it	60	Home, only after Milestone 9

THE RUBRIC (LESSON 13.2, REPRODUCED EXACTLY)

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Framing	"Predict X" with no decision or user.	Target and metric chosen, but the decision, the costs of errors or the prediction moment are vague.	Decision, user, target, prediction moment, split, metric and baseline written down before modeling, and followed.	Also states in advance what result would mean "do not deploy", and translates the metric into the business's units.
Data and leakage	Columns used as they come. No checks.	Shapes and missing values checked, but some feature's timing is never questioned.	Every feature's availability at prediction time is checked; transformations are fitted inside pipelines; the split matches the data's structure (time, groups).	Also demonstrates a leak caught (score before and after) and documents data quirks such as stockout zeros or mislabeled stars.
Evaluation	One score on training data, or accuracy on rare events.	Test score against a baseline, but the test set was used for choices, or the metric does not fit the decision.	Model selection with cross-validation on the training data, test used once, metric fits the decision, baseline beside it.	Also an interval (bootstrap) for the test metric and an operating point (threshold, top-k) chosen from costs or capacity.

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Modeling	One model, default settings, no comparison.	Several models tried in a loop, without a reason for the order or the final pick.	A simple model first, then a stronger one; feature work and modest tuning that each earn their place in CV.	Also shows which changes did not help and stops when gains are smaller than the noise.
Error analysis and explanation	None.	A feature importance chart, unexplained.	Errors broken down by a meaningful subgroup; worst cases read; drivers explained as properties of the model.	Also turns an error finding into a fix (a feature, a rule, a caveat) and measures its effect.
Responsible ML	Not considered.	A generic paragraph about fairness.	Every checklist item answered for this problem: who bears errors, subgroup results, sensitive attributes, privacy, human review, monitoring.	Also changes the system because of a finding (a threshold, a review step, a removed feature) and says why.
Communication	The notebook is the report.	A model card exists, but numbers lack baselines, or caveats are generic.	One to two pages that lead with a recommendation, with quantified, baseline-relative evidence, specific limitations and finding-first figures.	Also readable by a non-specialist, who could act on it correctly.
Reproducibility	Fails or gives different numbers when rerun.	Runs, but needs cells in a special order, absolute paths or unseeded randomness.	Restart and Run All works from the project folder; seeds set; model saved with skops; versions pinned; card numbers match the output.	Also a README, functions with clear names, and results logged (MLflow or results.json) for each experiment.

THE RESULT

- Score each row 1 to 4 (whole numbers). The lesson gives no weights: all eight rows count equally. Record the total out of 32 for your records only.
- **Successful capstone:** 3 (Proficient) or higher on every row, as the lesson states ("Proficient in every row is a successful capstone and the standard expected of a junior machine-learning engineer").
- **Successful with a revision:** exactly one row at 2 and none at 1. The learner fixes that row within one week; re-score that row only.
- **Not yet:** any row at 1, two or more rows at 2, or a notebook that does not run. Return written feedback (one thing done well per row and the single change that would most improve the work) and allow one resubmission within two weeks.
- **Reproducibility first:** run the notebook before reading anything else (Restart and Run All, from the project folder, with the course `data/` beside it). If it fails, ask for a fix within 48 hours before scoring the other rows.

SCORING GUIDANCE BY ROW

ROW	WHAT TO LOOK FOR
Framing	The first Markdown cell: decision, user, target, prediction moment, split, metric, baseline, written before modeling, and a notebook that follows it (or says where it changed). Exemplary: a pre-stated "do not deploy" bar and the metric in the business's units (frauds held per day, units of stock, complaints found).
Data and leakage	A feature-timing table (or equivalent sentences) for every column; all preprocessing inside pipelines; the template's split kept. Exemplary: a leak shown with scores before and after, and quirks documented (stockout zeros, contradictory stars, the drifting fraud rate).
Evaluation	Choices made with a harness on the training data; ONE test score next to the baseline on the same test set; a metric that fits the decision. Exemplary: a bootstrap interval (paired with the baseline, by week or customer where rows are not independent) and an operating point from capacity or costs.
Modeling	A simple pipeline first, then a stronger one; feature work and modest tuning that each earned their place (an attempts log with CV scores and spreads). Exemplary: the log shows what did NOT help, and work stopped when gains fell inside the noise.
Error analysis and explanation	Errors by a meaningful subgroup with the baseline beside them; the worst cases read; drivers described as what the model relies on. Exemplary: a finding turned into a fix (a rule, a feature, a caveat) and its effect measured.
Responsible ML	All ten checklist items answered specifically. Look hardest at "who bears the errors" and "sensitive attributes". Exemplary: the system changed because of a finding (a review step, a removed feature, a threshold).

ROW	WHAT TO LOOK FOR
Communication	Read `model_card.md` first, as the brief's reader would: recommendation first, numbers with baselines and intervals, specific limitations, finding-first figures, no causal words. A well-argued "do not deploy" is scored like a success.
Reproducibility	Restart and Run All; pick three numbers in the card and find them in the output; `model.skops` loads with a written trusted list; `requirements.txt` pins versions. Any card number you cannot find is a Developing mark and a question for the learner.

SCORING SHEET

Learner: _____ · Problem: A / B / C / D / E / own · Date received: _____ · On time: yes / no

ROW	SCORE 1–4	EVIDENCE (CELL OR LINE IN MODEL_CARD.MD)
Framing		
Data and leakage		
Evaluation		
Modeling		
Error analysis and explanation		
Responsible ML		
Communication		
Reproducibility		
Total (of 32) · Result: successful / successful with revision / not yet		

SUBMISSION AND ACADEMIC HONESTY

- **Deadline:** one week after Session 13, as a zipped project folder without the course `data/` folder, or a link to a repository.
- **AI assistants and code from the web** are allowed on the terms a working engineer accepts: the learner is responsible for every line and every claim. The README says what help was used and for what. Undisclosed help the learner cannot explain is treated as not their work.

- **The explain-two-cells check:** for any submission you may pick two cells and ask the learner to explain, in five minutes, what they do and why the result is right. A cell the learner cannot explain is revised; the row it affects is scored after the revision.
- **Shared ideas, not shared code:** learners on the same problem may compare splits and traps; notebooks, figures and wording are their own.
- **Data and files:** the README carries the lesson's line ("Pinecrest Outfitters is a fictional company; the data was written for the Python Machine Learning course by Ray de la Paz.") and links to `pinecrest-ml-data.zip` instead of republishing it. No `.pkl` or `.joblib` files for others to load: `.skops` or none. No real or work data.
- Do not share these notes with learners before or after grading; feedback refers to the rubric and to the learner's own work.

LATE WORK AND ALTERNATIVES

- **Extension:** one extra week on request before the deadline, no reason needed. Later requests at your discretion.
- **Late without an extension:** scored on the rubric as usual; record the date received. No points deducted: the rubric measures the work.
- **Missed Session 13:** the learner records a three-minute talk (screen and voice) about their one figure and joins a 15-minute check-in with you within the week.
- **Missed Sessions 12 and 13, or a narrower scope agreed in advance:** one metric, one baseline, one improved model, one subgroup table. All eight rows still apply, judged on the narrower plan.
- **Accessibility:** the showcase can be a written gallery card; the card can be read aloud to you. The analysis is the same.

RECORDING CHECKPOINT 1 ALONGSIDE THE CAPSTONE

Kit B has one formative checkpoint before the capstone: Checkpoint 1 in Session 6 (Modules 7 to 9, scored with its own five-row rubric in the Session 6 answer key). It is not weighted into the capstone.

LEARNER	CHECKPOINT 1 (S6) SCORE / STATUS	CAPSTONE PROBLEM	ROWS (FR, D, EV, MO, ER, R, CO, RE)	TOTAL /32	RESULT

- Record the checkpoint as its rubric score and a status, Met or Not yet; a Not yet may be redone once at home before the capstone deadline.
- **Course completion** = Checkpoint 1 Met (or redone) and a successful capstone.

- Use the checkpoint for coaching: a Not yet predicts trouble in Evaluation and Error analysis; visit those learners first in the Session 12 studio.

ANSWER NOTES · PROBLEM A: DEMAND FORECASTING

A SOUND PROJECT: STEPS

- Plan the decision as an order placed ahead of the week: features are what is known then (store, product, category, calendar, list price, planned discount and promo, holiday week). Actual weekly temperature is not.
- Keep the template split (2024 train with 5,088 rows, 2025 test with 4,896). Inside 2024, rolling-origin folds by week; note that they validate on unseen months and are pessimistic (Session 12 harness: per-product mean 13.92, ridge 13.13).
- Baselines: the template's per-product mean, plus per product × store and "same week last year".
- Models: one-hot + ridge, then gradient boosting with categorical product and store (optionally Poisson loss). A bootstrap interval resampling weeks. Errors by category, with the baseline beside them.

KEY NUMBERS (FROM THE INSTRUCTOR'S OWN RUN ON THE COURSE DATA)

- Baselines on 2025: per-product mean 13.42; per product × store 12.95; same week last year 13.81 (worse: one noisy week against a yearly average); the overall mean 17.13.
- Ridge on one-hot + numeric: 11.29. Gradient boosting (no temperature): 8.19; Poisson loss 8.12. Learning-rate tuning by rolling folds on 2024 lands on the default 0.1 (Session 13 review handout: 8.29 with a slightly smaller feature set).
- Units saved against the per-product mean: 5.23 (95% CI 4.91 to 5.53 resampling rows; 4.68 to 5.81 resampling the 51 weeks). Model MAE 95% CI 7.92 to 8.49. Over-forecast share 48.9% (balanced).
- By category (baseline → model): Footwear 18.7 → 12.5, Camp Kitchen 19.3 → 11.3, Snow 13.3 → 5.8; Snow in June to August sells 0.5 units a week and is forecast at 1.5.
- Zeros: 225 in all, 200 of them Snow in summer (real). A neighbor rule (zero with 12+ units on both sides) finds 11 suspicious zeros, 4 of them in 2025 (14 stockouts were planted).
- The confounding trap: 40%-off weeks average 9.7 units against 20.1 at full price, because deep discounts are off-season clearance.

The run that produced them:

```
5088 4896 2024-12-30 2025-01-06
zeros: 225  Snow zeros: 200
seasonal naive (same week 2024): MAE 13.81 on 4896 rows
per product x store mean: MAE 12.95
per-product mean (template): MAE 13.42

full price 20.1 units/week; 40% off 9.7 units/week (122 rows)
months with 40% off: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

ridge, no temperature          MAE 11.29
HGB, no temperature            MAE 8.19
HGB + actual temperature       MAE 8.16
HGB poisson, no temperature    MAE 8.12

HGB + last year's temperature for that week: MAE 8.18

TimeSeriesSplit(4) MAE by fold: [11.01, 11.91, 10.93, 12.02]

              e_base  e_model  units
category
Footwear        18.7     12.5   45.6
Camp Kitchen    19.3     11.3   36.9
Accessories     13.1      8.7   29.8
Apparel         12.6      8.0   25.6
Tents           11.3      6.6   18.4
Sleeping        10.7      6.3   19.3
Climbing         9.1      6.0   19.2
Snow            13.3      5.8   12.3
Packs            9.2      5.6   18.3
Snow in Jun-Aug: mean units 0.5, mean forecast 1.5
MAE saved vs per-product mean: 5.23 units (95% CI 4.91 to 5.53)
model MAE 8.19 (95% CI 7.92 to 8.49)
over-forecast share 48.9%

suspicious zeros (neighbours both 12+): 11 of which in 2025: 4
{'Camp Kitchen': 4, 'Footwear': 3, 'Accessories': 2, 'Tents': 2}
```

THE ANSWER THE DATA SUPPORT

Use the gradient-boosting forecast for store orders: it cuts the weekly error by about 5 units (8.2 against 13.4 for the per-product average, about 39% lower) at every category, with the largest absolute errors on the high-volume Footwear and Camp Kitchen lines. Limitations to state: one year of training history (one of each holiday), stockout zeros that understate demand, and a model that does not set prices (price effects are confounded with season).

TRAPS

- **Actual temperature as a feature.** It is unknown when the order is placed. On this data it barely matters (with actual temperature 8.16; with last year's temperature for that week 8.18; without 8.19), so the honest fix costs

nothing; the learner who keeps it has a Data and leakage problem regardless of the number.

- **A random split of weekly rows** (the Session 13 review handout): the random-split MAE is 7.60 to 7.74, which flatters the model against the 2025 truth (8.29).
- **Stockout zeros as demand.** They are outliers for a regression and they understate demand; mention them, optionally exclude them from training with a stated rule.
- **Reading price effects from raw averages** (40%-off weeks sell less): season, not price. A learner who claims an elasticity without product × month controls over-claims.
- **Comparing absolute MAE across categories** without volume or the baseline: Snow looks best only because it sells little.

HONEST NEGATIVE RESULTS (SCORE THEM AS WELL AS A SUCCESS)

- Temperature adds nothing measurable once the month is in: a clean, reportable negative.
- "Same week last year" is worse than a plain average: a baseline that sounds smart and is not.
- Rolling folds inside one year cannot validate the season: the harness's numbers (11 to 13) are higher than the 2025 test's; saying why is Exemplary Evaluation.

FOLLOW-UP QUESTIONS TO ASK THE LEARNER

- "Which columns do you know on the day the order is placed?"
- "Is every zero zero demand? How would you tell?"
- "Does the improvement hold in every category, against the same baseline?"

WHAT A WEAK SUBMISSION TYPICALLY GETS WRONG

- Uses actual temperature, or a random split, and reports the flattering number.
- Compares with the overall mean (17.13) instead of the template's baseline.
- Claims "discounts reduce sales" or an elasticity from raw averages.

ANSWER NOTES · PROBLEM B: FRAUD REVIEW QUEUE

A SOUND PROJECT: STEPS

- Keep the January-to-May / June split; validate inside it on May (train January to April). Accuracy is not a metric here.
- Baselines: no model (average precision = the fraud rate) and a one-line rule (rank by `velocity_24h`), at the team's capacity of 1% of June (83 payments).
- Models: scaled numerics + one-hot category + logistic regression; gradient boosting (defaults, then modest tuning); optionally IsolationForest as an unsupervised comparison.

- One look at June: average precision with a paired bootstrap interval, frauds held at 1% (and 2%, 5% if the learner explores capacity), false alarms by account age and hour.

KEY NUMBERS (FROM THE INSTRUCTOR'S OWN RUN ON THE COURSE DATA)

- June: 8262 payments, 64 frauds; monthly fraud rate 1.0% (January) to 0.77% (June); "nothing is fraud" accuracy 0.9923.
- June average precision / AUC / frauds in the top 1% (83 held): logistic regression 0.539 / 0.883 / 33; boosting defaults 0.477 / 0.895 / 32; tuned boosting (learning rate 0.05, 300 iterations, 15 leaves, 40 per leaf, l2 1.0, early stopping) 0.556 / 0.901 / 35; IsolationForest without labels 0.376 / 0.848 / 31. A random 1% holds 0.64 frauds; the velocity rule's top 1% holds 19 (average precision 0.250).
- Validation on May: logistic regression 0.539, boosting defaults 0.491, tuned 0.532: the validation month does not prefer boosting.
- Tuned boosting: AP 95% CI 0.423 to 0.668; minus defaults +0.079 (0.027 to 0.133); minus logistic regression 0.017 (-0.046 to 0.077).
- Capacity: hold 2% → 41 frauds (recall 0.64); 5% → 47 (0.73). Lesson 2.5's costs ($10 a review, $375 a missed fraud): June costs $24,000 with no reviews, $11,705 holding 1%, $10,285 at 2%, $9,865 at the break-even probability 0.027 (199 held: more than the team's capacity).
- Who bears the errors: new accounts (under 30 days) are 8.4% of legitimate June payments but 29.2% of the 48 false alarms; night payments 3.6% of legitimate, 29.2% of false alarms.

The run that produced them:

```
41738 8262 374 64
monthly fraud rate: {1: 1.0, 2: 0.99, 3: 0.87, 4: 0.87, 5: 0.76, 6: 0.77}
June accuracy of 'nothing is fraud': 0.9923

logistic regression          AP 0.539  AUC 0.883  top 1%: 33 of 83 held are fraud (reca
HGB defaults                 AP 0.477  AUC 0.895  top 1%: 32 of 83 held are fraud (reca
HGB tuned (SHELL settings)   AP 0.556  AUC 0.901  top 1%: 35 of 83 held are fraud (reca
IsolationForest (no labels)  AP 0.376  AUC 0.848  top 1%: 31 of 83 held are fraud (reca
June frauds 64, capacity 83; a random 1% holds 0.64 frauds

validate on May: LR            AP 0.539
validate on May: HGB defaults  AP 0.491
validate on May: HGB tuned     AP 0.532

random 70/30 split, tuned HGB: AP 0.506  AUC 0.864

tuned: new accounts 8.4% of legit June payments, 29.2% of false alarms (48 false alarms
tuned: night payments 3.6% of legit, 29.2% of false alarms
tuned AP 0.556 (95% CI 0.423 to 0.668)
tuned minus defaults AP: 0.079 (95% CI 0.027 to 0.133)
tuned minus LR AP: 0.017 (95% CI -0.046 to 0.077)
hold 2%: 166 held, 41 fraud, recall 0.64
hold 5%: 414 held, 47 fraud, recall 0.73

June cost, review nothing: 24000
June cost, hold top 1%: 11705
June cost, hold top 2%: 10285
June cost, hold top 5%: 10515
June cost, hold everyone above break-even p=0.027: 9865 (199 held)
```

THE ANSWER THE DATA SUPPORT

Hold the riskiest 1% of payments for review each day: on June that catches 33 to 35 of 64 frauds with either logistic regression or tuned boosting (about half the fraud, against under one for a random 1%), and the two models cannot be told apart on one month. Recommend the simpler model unless boosting wins on more months, keep holds as reviews (never automatic declines), and give new accounts a fast release path because they bear a third of the false alarms. A capacity of about 2% would catch about two thirds of the fraud and is cheaper under the course's cost figures: a business decision worth putting to the team.

TRAPS

- **Accuracy** (0.9923 for doing nothing).
- **A stratified random split** mixing months: it hides the drift and gives a different number (tuned boosting AP 0.506, AUC 0.864); the card must use June.
- **Resampling (SMOTE, undersampling) before the split,** or outside an imblearn pipeline.
- **Declaring boosting the winner** from 0.556 vs 0.539 without an interval.

- **Removing the account-age feature** (`account_age_days`) to protect new customers without measuring the effect on every group.

HONEST NEGATIVE RESULTS (SCORE THEM AS WELL AS A SUCCESS)

- Tuned boosting is not distinguishable from logistic regression on June (difference interval includes 0), and May validation prefers neither: the Modeling row rewards saying so and choosing on other grounds.
- About a third of frauds look like normal purchases: recall at sensible capacity tops out around two thirds. "No model catches them all" is a finding, not a failure.
- The unsupervised IsolationForest is clearly worse (0.376): fine to report as a tried-and-rejected idea.

FOLLOW-UP QUESTIONS TO ASK THE LEARNER

- "How many payments can the team look at, and how many frauds does that hold?"
- "Which month chose your model, and which month tests it?"
- "Who is kept waiting by a false alarm, and what happens to them?"

WHAT A WEAK SUBMISSION TYPICALLY GETS WRONG

- Leads with accuracy or with AUC only.
- Tunes on June.
- No capacity, no subgroup analysis of the false alarms.

ANSWER NOTES · PROBLEM C: REVIEW TRIAGE

A SOUND PROJECT: STEPS

- Text only (or text + title, with the choice stated). TF-IDF + logistic regression first; vectorizer inside the pipeline; stratified CV with accuracy AND negative recall.
- Improvements: bigrams, `class_weight="balanced"`, or a threshold on P(negative) chosen on out-of-fold predictions for a target negative recall.
- Test once: accuracy with an interval, negative recall, and the share of reviews flagged at the operating point. Errors with "not" vs without; by category. Read the most confident errors.
- Measure (and caveat) what the model does with the 905 3-star reviews it never saw.

KEY NUMBERS (FROM THE INSTRUCTOR'S OWN RUN ON THE COURSE DATA)

- Test set: 774 reviews, 74.5% positive; always positive 0.745 with negative recall 0.
- TF-IDF + LR: CV 0.857, test 0.877, negative recall 0.660; English stop words (removes "not") test 0.864; 1-2 grams 0.877 / 0.655; 1-2 grams balanced CV 0.875, test 0.873, negative recall 0.873.

- Threshold for 90% negative recall chosen on CV: P(negative) ≥ 0.235; on the test set negative recall 0.909, 36.8% of reviews flagged, precision 0.628. Accuracy 95% CI (1-2 grams) 0.855 to 0.901.
- Titles alone: 0.969; title + text 0.937 (only 167 distinct titles in training: short phrases that restate the rating).
- 3-star reviews: 20.8% predicted negative at 0.5, 64.8% at the 90%-recall threshold. Accuracy with "not" in the text 0.813 (134 reviews) vs 0.891 without; the weakest categories Sleeping (0.800) and Camp Kitchen (0.822).
- Errors: 95 of 774, only 3 with confidence above 0.9; the confident ones are 2-star reviews whose text reads positive ("It is built to last."): stars that contradict the text.

The run that produced them:

```
2321 774 positive share test 0.745
tfidf + LR                   CV acc 0.857  test acc 0.877  negative recall 0.660
tfidf stop_words='english'   CV acc 0.851  test acc 0.864  negative recall 0.624
tfidf 1-2 grams, min_df=2    CV acc 0.860  test acc 0.877  negative recall 0.655
tfidf 1-2 grams, balanced    CV acc 0.875  test acc 0.873  negative recall 0.873

title only: test acc 0.969
title + text: test acc 0.937  negative recall 0.858
distinct titles: 167 of 2321

P(negative) threshold for 90% negative recall on CV: 0.235
test at that threshold: negative recall 0.909, share of reviews flagged 0.368, precisio
test accuracy 95% CI 0.855 to 0.901

3-star reviews: 905; predicted negative at 0.5: 20.8%; flagged at the 90%-recall thresh
errors: 95 | confident (>0.9): 3
2 stars | Fell short | It is built to last.
2 stars | Fell short | Used it mostly on weekend trips. It shed rain all night. It goes
2 stars | Returned it | Got it during the spring sale. It showed up two days early. Alt
2 stars | Not worth it | It arrived late. Although it is a great value.
accuracy with 'not' in the text 0.813 (134 reviews) vs without 0.891
              size   mean
category
Sleeping        45  0.800
Camp Kitchen    45  0.822
Accessories    111  0.847
```

THE ANSWER THE DATA SUPPORT

Use the model to order the product team's queue, not to hide anything: with balanced weights (or a threshold set for 90% negative recall) it finds about nine in ten complaints while flagging about a third of all reviews, against none for the current "read them in order". Limitations: 3-star reviews are unvalidated (a fifth to two thirds would be flagged, depending on the threshold), negation ("not waterproof at all") is the main weakness, and some star ratings contradict their own text.

TRAPS

- **Accuracy only** (0.877 sounds good while a third of complaints are missed).
- **Stop words** from scikit-learn's English list: they remove "not".
- **Fitting the vectorizer on all reviews** before the split.
- **Titles without saying so:** 0.969 is a near-restatement of the rating, legitimate only if titles arrive with the review; the card must state it.
- **Blaming the model for contradictory stars** without reading the errors.

HONEST NEGATIVE RESULTS (SCORE THEM AS WELL AS A SUCCESS)

- Bigrams do not help on this small vocabulary (0.877 either way): a tried-and-rejected change.
- The accuracy interval (about ±0.02) is wider than most differences between variants: rank them by negative recall and the business cost, not by the third decimal.

FOLLOW-UP QUESTIONS TO ASK THE LEARNER

- "Which error is the costly one, and what is your recall of it?"
- "What will the live system do with a 3-star review?"
- "Read me your three most confident mistakes."

WHAT A WEAK SUBMISSION TYPICALLY GETS WRONG

- Reports accuracy without negative recall.
- Drops 3-star reviews and never mentions them again.
- Uses titles silently.

ANSWER NOTES · PROBLEM D: RECOMMENDATIONS

A SOUND PROJECT: STEPS

- Keep leave-last-out for the test (3,000 customers, one hidden purchase each); carve a second hidden purchase from the training rows for validation; mask owned products everywhere.
- Baselines: random (about 0.05) and most popular unowned (0.209). Models: item-kNN (cosine on the binary matrix), truncated SVD with k chosen on validation, content-based on `primary_activity` (with popularity as a tie-break).
- Test once: hit rate at 10 with a paired bootstrap against popularity; by history length; coverage (distinct products recommended) and popularity bias.
- Cold start in the card: what a new customer and a new product see.

KEY NUMBERS (FROM THE INSTRUCTOR'S OWN RUN ON THE COURSE DATA)

- Test hit rate at 10: random 0.047, popularity 0.209, item-kNN 0.308, SVD k = 5 0.311.

- Validation (second-to-last purchase): SVD k = 3 / 4 / 5 / 6 / 8 / 10 / 20 → 0.244 / 0.261 / 0.285 / 0.277 / 0.258 / 0.244 / 0.216; item-kNN 0.286; popularity 0.209. Validation picks k = 5, the same as the lesson's test curve.
- Item-kNN minus popularity 0.099 (95% CI 0.082 to 0.116); SVD k = 5 minus item-kNN 0.003 (-0.009 to 0.016).
- By history length (popularity → item-kNN): 2 to 4 purchases 0.199 → 0.257; 16+ 0.246 → 0.382.
- Coverage: popularity recommends 31 distinct products (99.8% of slots on the 20 best sellers); item-kNN 161 (51.5%); SVD 153.
- Content-based on activity: 0.176; with a popularity tie-break 0.272. Unmasked item-kNN: 0.113 (69.1% of slots already owned). A random hidden item instead of the latest: item-kNN 0.299.

The run that produced them:

```
random       HR@10 0.047
popularity   HR@10 0.209
item-kNN     HR@10 0.308
SVD k=5      HR@10 0.311

validation HR@10 by k: {3: 0.244, 4: 0.261, 5: 0.285, 6: 0.277, 8: 0.258, 10: 0.244, 20
validation item-kNN 0.286  popularity 0.209

content (activity)             HR@10 0.176
content + popularity tiebreak  HR@10 0.272

      popularity  item-kNN  SVD k=5  customers
2-4        0.199     0.257    0.267        607
5-8        0.202     0.280    0.280        987
9-15       0.204     0.334    0.338        950
16+        0.246     0.382    0.379        456
popularity distinct products recommended 31 of 200; share of slots on the 20 best sell
item-kNN   distinct products recommended 161 of 200; share of slots on the 20 best sel
SVD k=5    distinct products recommended 153 of 200; share of slots on the 20 best sel
item-kNN minus popularity: 0.099 (95% CI 0.082 to 0.116)
SVD k=5 minus item-kNN: 0.003 (95% CI -0.009 to 0.016)

unmasked item-kNN (diagonal kept): HR@10 0.113; share of slots already owned 69.1%
random-item split: item-kNN HR@10 0.299
```

THE ANSWER THE DATA SUPPORT

Replace the best-seller row with item-based recommendations: the customer's next purchase is in the list about 31% of the time against 21%, for customers with short and long histories alike, and the list covers 161 products instead of 31. SVD is a statistical tie and harder to explain; item-kNN's "because you bought X" is easier to show. Limitations: brand-new customers still get best sellers, a never-bought product can only be reached by content (which alone is weaker than popularity), and purchase histories are personal data.

TRAPS

- **Choosing k on the test set** (Lesson 11.2 shows the test curve for teaching; a capstone must use the validation item).
- **Forgetting the owned-product mask:** 0.113 instead of 0.308.
- **Evaluating on training purchases** or building the matrix from all rows.
- **Calling a 0.003 SVD gain a win.**
- **Publishing example customer lists** (purchase histories) in figures.

HONEST NEGATIVE RESULTS (SCORE THEM AS WELL AS A SUCCESS)

- SVD does not beat item-kNN (interval includes 0).
- Content-based alone loses to popularity (0.176 vs 0.209): report it, and keep it for cold start.
- On this data the random-item split is barely different (0.299 vs 0.308) because purchase dates are spread at random; the latest-item split is still the right design, and a learner who notices this and says why scores Exemplary in Data and leakage.

FOLLOW-UP QUESTIONS TO ASK THE LEARNER

- "Which purchase chose your k?"
- "What does a brand-new customer see, and a product nobody has bought?"
- "How much of your list is best sellers?"

WHAT A WEAK SUBMISSION TYPICALLY GETS WRONG

- No mask; k from the test set.
- Only the average hit rate: no history-length or coverage view.
- A model card that shows individual customers' lists.

ANSWER NOTES · PROBLEM E: CUSTOMER SEGMENTS

A SOUND PROJECT: STEPS

- Behavior columns only (the template's four: `orders_12m`, `avg_order_value`, `discount_share`, `return_rate`), scaled; logs of the skewed two are a reasonable variant.
- Choose k with several signals: silhouette by k, the inertia elbow, Gaussian-mixture BIC, and stability (ARI across seeds and between a model fitted on half the data and the full model).
- Profiles per segment (means of the four features and of `app_user`), names a marketer recognizes, and a churn check AFTER clustering (never trained on). `age_group` kept out; segments checked by age afterwards.

KEY NUMBERS (FROM THE INSTRUCTOR'S OWN RUN ON THE COURSE DATA)

- Baseline: loyalty tiers silhouette 0.079; five random groups -0.023.

- K-Means, scaled, silhouette by k = 2..8: 0.271, 0.35, 0.413, 0.47, 0.433, 0.402, 0.353; log + scaled peaks at 5 too (0.478); the two k = 5 solutions agree (ARI 0.920).
- Stability at k = 5: ARI 1.000 over ten seeds; a model fitted on a random half agrees with the full one at 0.986 to 0.996. k = 4 is unstable (seed ARI down to 0.512); k = 6 0.952.
- Gaussian-mixture BIC: 3 → 48,267, 4 → 45,391, 5 → 43,175, 6 → 42,990, 7 → 42803: BIC alone would keep going past 5.
- Profiles at k = 5 (n, orders, average order value, discount share, return rate, app users, churn): 1,331 / 9.3 / $89 / 0.10 / 0.04 / 54% / 3%; 812 / 6.3 / $165 / 0.18 / 0.28 / 49% / 10%; 1,073 / 2.5 / $337 / 0.11 / 0.05 / 31% / 16%; 1,522 / 2.1 / $98 / 0.13 / 0.06 / 24% / 19%; 1,262 / 5.3 / $66 / 0.51 / 0.08 / 47% / 0.32. Reasonable names: frequent regulars, gear testers (returns), big-trip planners, casual campers, deal hunters.
- Unscaled: silhouette 0.545 with sizes 286 to 2,792 and ARI 0.267 against the scaled solution; all 11 numeric columns: silhouette 0.158 to 0.189; adding `age_group` one-hot barely moves the segments (ARI 0.999) but puts a sensitive attribute into the targeting.

The run that produced them:

```
scaled {2: 0.271, 3: 0.35, 4: 0.413, 5: 0.47, 6: 0.433, 7: 0.402, 8: 0.353}
log + scaled {2: 0.298, 3: 0.368, 4: 0.427, 5: 0.478, 6: 0.45, 7: 0.392, 8: 0.344}
baseline tiers 0.079

ARI vs seed 42 over 10 seeds: min 1.000, mean 1.000
ARI, model fit on a random half then applied to all: [0.995, 0.992, 0.996, 0.996, 0.98
k=4: seed ARI min 0.512
k=6: seed ARI min 0.952
GMM BIC: {3: 48267, 4: 45391, 5: 43175, 6: 42990, 7: 42803}

            n  orders     aov  discount  returns   app  churn
segment
2        1331    9.29   88.76      0.10     0.04  0.54   0.03
0         812    6.26  164.64      0.18     0.28  0.49   0.10
4        1073    2.48  336.92      0.11     0.05  0.31   0.16
1        1522    2.12   97.98      0.13     0.06  0.24   0.19
3        1262    5.32   65.82      0.51     0.08  0.47   0.32

unscaled: silhouette 0.545; cluster sizes [286, 620, 686, 1616, 2792]; ARI with scaled
all 11 numeric columns, silhouette by k: {3: 0.158, 4: 0.172, 5: 0.189, 6: 0.174}
with age one-hot: ARI with behaviour-only segments 0.999
log + scaled vs scaled k=5: ARI 0.920
```

THE ANSWER THE DATA SUPPORT

Five stable behavior segments describe customers far better than the loyalty tiers (silhouette 0.47 against 0.08) and survive another seed and half the data. They differ sharply in churn (3% to 32%) though churn was never used,

which makes them useful for message planning; the card says they describe behavior, not causes, and that age was left out and checked afterwards.

TRAPS

- **No scaling** (a higher silhouette from a clustering dominated by order value).
- **All numeric columns** thrown in (silhouette drops to under 0.2).
- **k from one signal** (BIC keeps falling; the elbow alone is vague).
- **Demographics in the clustering.**
- **Causal names** ("deal hunters leave because of discounts").

HONEST NEGATIVE RESULTS (SCORE THEM AS WELL AS A SUCCESS)

- BIC does not pick 5: a reported disagreement between signals is good science.
- The four-segment solution is unstable: say why k = 4 was rejected.
- Log transforms change little (ARI 0.92 with the raw-scaled solution): a tried-and-rejected idea.

FOLLOW-UP QUESTIONS TO ASK THE LEARNER

- "Which column dominates your distances?"
- "Do your segments survive another seed and half the data?"
- "Would a marketer recognize each profile from its numbers alone?"

WHAT A WEAK SUBMISSION TYPICALLY GETS WRONG

- Silhouette only.
- Segments named before profiles are computed.
- Churn used in the clustering (then the churn check means nothing).

REFERENCE: THE CAPSTONE LABS' OUTPUTS (SESSIONS 12 AND 13)

The two Session 12 labs print the second baselines and the training-data harness numbers that the notes above rely on:

```
                                          metric   value
problem  baseline
demand   per-product mean                MAE (units)  13.419
         per product x store mean        MAE (units)  12.953
         same week last year             MAE (units)  13.810
fraud    no model                  average precision   0.008
         rule: velocity_24h        average precision   0.250
reviews  always positive                    accuracy   0.745
         keyword rule                       accuracy   0.788
recs     most popular unowned         hit rate at 10   0.209
         random unowned               hit rate at 10   0.053
segments existing loyalty tiers           silhouette   0.079
         five random groups               silhouette  -0.023

fraud rule, top 1% of June (83 payments): 19 of 64 frauds
reviews keyword rule: negative recall 0.477 (always positive: 0.000)
```

```
                                    score  second
problem  candidate
demand   per-product mean           13.924   1.986
         ridge, one-hot             13.128   2.330
fraud    no model                    0.008     NaN
         logistic regression         0.539     NaN
reviews  always positive             0.745   0.000
         tf-idf + logistic regression  0.857   0.577
recs     popularity                  0.209     NaN
         item-kNN                    0.286     NaN
         SVD k=3                     0.245     NaN
         SVD k=5                     0.285     NaN
         SVD k=10                    0.248     NaN
segments existing loyalty tiers      0.079     NaN
         k-means k=4                 0.413   0.999
         k-means k=5                 0.470   1.000
         k-means k=6                 0.433   0.953

harness per problem: {'demand': 'MAE, 4 rolling folds (lower is better)', 'fraud': 'ave
segments k=5: model fitted on half the customers agrees with the full model, ARI 0.995
```